

AWS DATA ENGINEER ASSOCIATE

DAY 2: Transformation & ETL/ELT — Comprehensive Study Guide

Exam Domain	Duration	Topics	Hands-On Labs
Data Ingestion & Transformation	1 Day (2 Hours Theory + 2 Hours Labs)	Glue Jobs, EMR/Spark Athena, Data Formats	4 Labs (1 per topic)

Learning Objectives

By the end of Day 2, participants will be able to:

- Design and execute AWS Glue ETL jobs using both visual interface and Python scripts
- Understand EMR cluster architecture and write Spark transformations using PySpark
- Query data in Amazon S3 using Athena with cost-optimization best practices
- Select the appropriate data format (Parquet, ORC, Avro, JSON, CSV) for a given workload
- Apply ETL vs ELT patterns and know when to use each approach on AWS

TOPIC 1 | AWS Glue Jobs & ETL/ELT

What is AWS Glue?

AWS Glue is a fully managed, serverless ETL (Extract, Transform, Load) service. It eliminates the need to provision or manage infrastructure — you simply define your transformation logic and Glue handles execution at scale using Apache Spark under the hood.

Core components:

Glue Data Catalog: Centralised metadata repository — stores table definitions, schemas, and partition info. Acts as the Hive Metastore for Athena and EMR.

Glue Crawlers: Automatically scan data sources, infer schemas, and populate the Data Catalog. Support S3, RDS, DynamoDB, and JDBC sources.

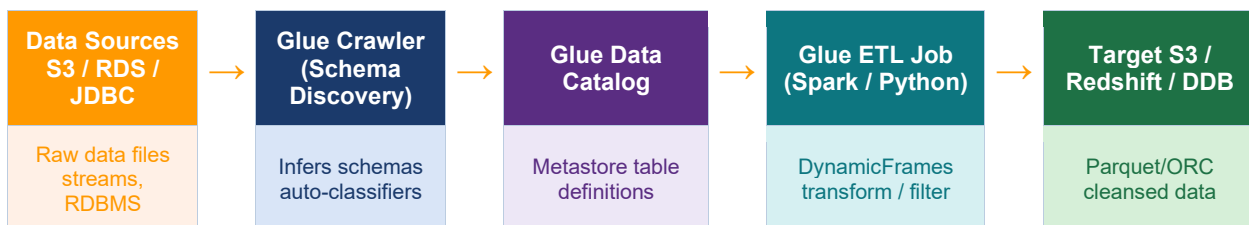
Glue ETL Jobs: Serverless Spark-based scripts (Python/Scala) that read, transform, and write data. Can be authored visually or as code.

Glue Workflows: Orchestration of multiple crawlers and jobs into a DAG-based pipeline with triggers.

Glue DataBrew: No-code visual data preparation tool with 250+ built-in transformations — ideal for analysts.

Architecture Diagram — Glue ETL Pipeline

The diagram below shows the end-to-end flow from raw data sources through Glue to a target data store:



Key Concept: Glue uses DynamicFrames (an extension of Spark DataFrames) which handle schema inconsistencies gracefully — e.g., columns that appear in some records but not others.

ETL vs ELT on AWS

Aspect	ETL (Extract-Transform-Load)	ELT (Extract-Load-Transform)
When to use	Schema well-known; sensitive data; limited target compute	Large-scale analytics; elastic target (Redshift/Athena)
Transformation location	Glue job (before writing to target)	Inside Redshift SQL / Athena / Spark after loading raw
Typical flow	S3 raw → Glue job → S3 curated/Redshift	S3 raw → Redshift COPY → Redshift SQL transforms
Latency	Higher (transform before load)	Lower initial load; transform on-demand
AWS services	Glue ETL, EMR	Redshift, Athena, dbt on Redshift
Data volume	Small–medium preferred	Large-scale, petabyte-range

Glue Job Types

Job Type	Runtime	Use Case	DPUs	Notes
Spark ETL	Apache Spark 3.x	Large-scale batch transforms	2–100	Default; auto-scales
Python Shell	Python 3.9	Lightweight tasks, API calls	0.0625 or 1	Low cost; no Spark overhead
Streaming ETL	Spark Streaming	Kinesis/Kafka transforms	2+	Micro-batch; continuous
Glue Studio Visual	No-code Spark	Drag-and-drop pipeline	2+	Generates PySpark code
Ray	Ray framework	ML/AI data prep	2+	New; GPU-friendly

Key Glue Concepts for the Exam

- DynamicFrame vs DataFrame: DynamicFrames tolerate schema inconsistencies; use toDF() to convert to Spark DF for complex transforms.
- Job Bookmarks: Track processed data — prevents reprocessing files already consumed. Enable on S3-source jobs.
- Connections: Glue Connections store JDBC credentials and VPC settings for RDS/Redshift access.
- Partitioning output: Use partitionKeys parameter in write_dynamic_frame to partition S3 output.
- Worker types: G.1X (4 vCPU/16GB), G.2X (8 vCPU/32GB), G.025X (lightweight). Choose based on memory needs.
- Glue triggers: Scheduled (cron), On-demand, or Event-based (job completion triggers downstream job).

LAB 1: Build a Glue ETL Job (Python/Spark)

Hands-On Exercise

Objective

Create a Glue crawler, catalog the raw data, build a Glue ETL job that transforms CSV to Parquet with partitioning, and verify output in S3.

Pre-requisites

- AWS account with Glue, S3 permissions
- S3 bucket: s3://your-bucket/raw/ (upload sample CSV below)

- IAM role: AWSGlueServiceRole with S3 access

Step 1 — Upload Sample Data to S3

Create a file `sales_data.csv` and upload to `s3://your-bucket/raw/sales/`

```
# sales_data.csv
order_id,customer_id,product,amount,order_date,region
1001,C001,Laptop,1200.00,2024-01-15,US-EAST
1002,C002,Mouse,25.50,2024-01-15,US-WEST
1003,C001,Keyboard,75.00,2024-01-16,US-EAST
1004,C003,Monitor,350.00,2024-01-16,EU-WEST
1005,C004,Laptop,1200.00,2024-01-17,AP-SOUTH

# Upload via CLI:
aws s3 cp sales_data.csv s3://your-bucket/raw/sales/sales_data.csv
```

Step 2 — Create and Run a Glue Crawler

1. Go to AWS Glue Console > Crawlers > Add Crawler
2. Name: `sales-raw-crawler` | Data source: S3 path `s3://your-bucket/raw/sales/`
3. IAM role: `AWSGlueServiceRole` | Target database: `sales_db` (create new)
4. Run crawler — it infers schema and creates table `sales` in `sales_db`
5. Verify in Glue Data Catalog > Tables: confirm column names and types

Step 3 — Create Glue ETL Job (Script)

Go to Glue > Jobs > Add Job > Spark script. Paste the following PySpark script:

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job
from pyspark.sql.functions import col, upper, to_date

args = getResolvedOptions(sys.argv, ['JOB_NAME'])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# Read from Glue Data Catalog
dyf = glueContext.create_dynamic_frame.from_catalog(
    database='sales_db',
    table_name='sales'
)

# Convert to Spark DataFrame for transformations
```

```

df = dyf.toDF()

# Transformations
df_clean = df \
    .filter(col('amount') > 0) \
    .withColumn('region', upper(col('region'))) \
    .withColumn('order_date', to_date(col('order_date'), 'yyyy-MM-dd')) \
    .withColumn('year', col('order_date').substr(1,4))

# Write as Parquet, partitioned by region and year
df_clean.write \
    .mode('overwrite') \
    .partitionBy('region', 'year') \
    .parquet('s3://your-bucket/curated/sales/')

job.commit()

```

Step 4 — Configure and Run

1. Worker type: G.1X | Number of workers: 2
2. Enable Job Bookmarks: Enable
3. Run job and monitor in Glue > Jobs > Run Details
4. Check CloudWatch Logs for errors

Step 5 — Verify Output

```

# Check partitioned output in S3
aws s3 ls s3://your-bucket/curated/sales/ --recursive

# Expected structure:
# s3://your-bucket/curated/sales/region=US-EAST/year=2024/part-00000.parquet
# s3://your-bucket/curated/sales/region=EU-WEST/year=2024/part-00000.parquet

```

Exam Tip: Job Bookmarks prevent reprocessing. If you re-run the crawler on the same source, Glue only processes new/changed files. This is a common exam scenario.

TOPIC 2 | Amazon EMR & Apache Spark

What is Amazon EMR?

Amazon EMR (Elastic MapReduce) is a managed big data platform that runs distributed processing frameworks like Apache Spark, Hive, HBase, Flink, and Presto on dynamically scalable EC2 clusters or serverless compute. It is the go-to service for large-scale data transformations that exceed Glue's capabilities.

Architecture Diagram — EMR Spark Pipeline



CSV, JSON, raw
logs

YARN / Hadoop
resource mgr

Core / Task nodes
auto-scaling

Joins, aggs,
window fns

Columnar,
partitioned

EMR Cluster Node Types

Node Type	Role	Can Be Spot?	Notes
Master (Primary)	Runs YARN ResourceManager, HDFS NameNode	No — must be On-Demand	Single point of control; use r5 instance family
Core	Runs HDFS DataNode + YARN NodeManager	Cautiously (data loss risk)	Stores HDFS data; termination can cause data loss
Task	Additional compute only (no HDFS)	Yes — Spot recommended	Good for burst capacity; no storage responsibility

Spark Core Concepts

RDD (Resilient Distributed Dataset): Immutable distributed collection. Low-level API. Use DataFrames/Datasets instead unless you need fine-grained control.

DataFrame / Dataset: Higher-level abstraction. Catalyst optimizer generates an efficient execution plan. Preferred for ETL work.

Spark SQL: SQL interface over DataFrames. Register a temp view and query with standard SQL syntax.

Transformations vs Actions: Transformations (map, filter, join) are lazy — no execution until an action (count, show, write) is called.

Partitioning: Data is split across partitions distributed to executors. Use repartition() or coalesce() to control output file count.

Catalyst Optimizer: Spark's built-in query planner — pushes filters down, reorders joins, eliminates redundant projections automatically.

Glue vs EMR — Decision Guide

Criteria	AWS Glue	Amazon EMR
Management	Fully serverless	Managed cluster (or EMR Serverless)
Scale	Up to ~100 DPUs	Thousands of cores
Custom libraries	Limited	Full — pip install anything
Startup time	1–2 min	3–7 min (cluster spin-up)
Cost model	DPU-hours	EC2 instance-hours + EMR premium
Data Catalog integration	Native	Via Hive Metastore connector
When to choose	Standard ETL, catalog integration	Complex ML, custom frameworks, massive scale

LAB 2: PySpark Transformations on EMR (or Glue Spark)

Hands-On Exercise

This lab can be run on EMR (via EMR Studio / Jupyter) or on Glue using a Spark script. The PySpark code is identical.

Scenario

You have raw web clickstream data in JSON format on S3. Transform it: parse timestamps, sessionise events, compute session duration, and write as partitioned Parquet.

Step 1 — Create Sample Input Data

```
# clickstream_raw.json (newline-delimited JSON)
{"user_id":"U001","event":"page_view","page":"/home","ts":"2024-03-01T09:00:00Z"}
{"user_id":"U001","event":"click","page":"/products","ts":"2024-03-01T09:02:30Z"}
{"user_id":"U002","event":"page_view","page":"/home","ts":"2024-03-01T09:01:00Z"}
{"user_id":"U001","event":"purchase","page":"/checkout","ts":"2024-03-01T09:08:00Z"}
{"user_id":"U002","event":"click","page":"/about","ts":"2024-03-01T09:05:00Z"}
```

```
aws s3 cp clickstream_raw.json s3://your-bucket/raw/clickstream/
```

Step 2 — PySpark Transformation Script

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, to_timestamp, window, count,
    min as spark_min, max as spark_max,
    unix_timestamp, round as spark_round
)
from pyspark.sql.window import Window

spark = SparkSession.builder.appName('ClickstreamETL').getOrCreate()

# Read JSON from S3
df = spark.read.json('s3://your-bucket/raw/clickstream/')

# Parse timestamp and add date partition column
df = df.withColumn('event_ts', to_timestamp(col('ts'), "yyyy-MM-dd'T'HH:mm:ss'Z'")) \
    .withColumn('event_date', col('event_ts').cast('date'))

# Compute session metrics per user per day
session_df = df.groupBy('user_id', 'event_date').agg(
    count('*').alias('total_events'),
    spark_min('event_ts').alias('session_start'),
    spark_max('event_ts').alias('session_end')
).withColumn('session_duration_mins',
    spark_round((unix_timestamp('session_end') - unix_timestamp('session_start')) / 60,
2)
)

# Show results
session_df.show(truncate=False)

# Write partitioned Parquet to S3
session_df.write \
    .mode('overwrite') \
```

```
.partitionBy('event_date') \
.parquet('s3://your-bucket/curated/sessions/')

spark.stop()
```

Step 3 — Performance Optimisation

- Use `.cache()` or `.persist()` on DataFrames that are reused multiple times in the same job
- Avoid wide shuffles (`groupBy`, `join`) on large datasets without pre-partitioning on join key
- Use broadcast joins for small lookup tables: `spark.sql('/*+ BROADCAST(dim) */ SELECT...')`
- Set `spark.sql.shuffle.partitions=200` (default) — reduce for small datasets to avoid overhead

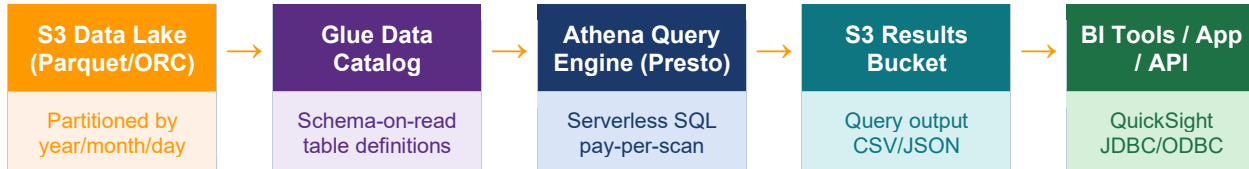
Exam Tip: EMR Spot instances save 60–90% cost for Task nodes. Never use Spot for Master or Core nodes. EMR gracefully handles Spot interruptions on Task nodes only.

TOPIC 3 | Amazon Athena Querying

What is Amazon Athena?

Amazon Athena is a serverless, interactive query service that lets you analyse data in Amazon S3 using standard SQL. It is built on open-source Presto/Trino and requires no infrastructure setup — you pay only for the data scanned per query (\$5 per TB scanned).

Architecture Diagram — Athena Query Flow



Key Features & Exam Points

- Schema-on-read: Athena does not load data — it applies schema at query time. Raw data stays in S3.
- Pricing: \$5/TB scanned. Columnar formats (Parquet/ORC) + compression can reduce cost by 85–95%.
- Supported formats: CSV, TSV, JSON, Parquet, ORC, Avro, Ion. Parquet/ORC strongly recommended.
- Partitioning: Use `MSCK REPAIR TABLE` or `ALTER TABLE ADD PARTITION` to register new partitions.
- Query result location: All results written to S3 results bucket. Configure in Workgroup settings.
- Workgroups: Isolate queries, enforce result bucket, set data scan limits, separate cost tracking.
- Athena Federated Query: Query data beyond S3 — RDS, DynamoDB, Redshift, on-prem via Lambda connectors.
- CTAS (Create Table As Select): Create a new table from query results. Useful for pre-aggregating data.
- Views: Virtual tables. No data movement. Query views like regular tables.
- Iceberg / Hudi: Athena v3 supports Apache Iceberg for ACID transactions, time travel, schema evolution.

Cost Optimisation Strategies

Strategy	Impact	Implementation
Use Parquet/ORC	Up to 87% cost reduction	Convert CSV/JSON to Parquet in Glue before querying

Strategy	Impact	Implementation
Partition pruning	Significant scan reduction	Filter on partition columns in WHERE clause
Column pruning	Reduces scan for columnar formats	SELECT only needed columns, avoid SELECT *
Compression (Snappy/gzip)	30–60% size reduction	Set compression in Glue job output
Partition projection	Eliminates Glue Catalog calls	Define partition ranges in table properties
Result reuse	Avoid repeated scans	Enable query result reuse (5 min window)

LAB 3: Athena Querying & Optimization

Hands-On Exercise

Objective

Create an Athena database over the Parquet data written in Lab 1, run queries, compare costs between CSV and Parquet, and use partition pruning.

Step 1 — Create Database and Table in Athena

Open Athena Query Editor. Set query result location to `s3://your-bucket/athena-results/`

```
-- Create database
CREATE DATABASE IF NOT EXISTS sales_analytics;

-- Create external table over Parquet with partitions
CREATE EXTERNAL TABLE IF NOT EXISTS sales_analytics.sales_curated (
    order_id      INT,
    customer_id   STRING,
    product       STRING,
    amount        DOUBLE,
    order_date    DATE
)
PARTITIONED BY (region STRING, year STRING)
STORED AS PARQUET
LOCATION 's3://your-bucket/curated/sales/'
TBLPROPERTIES ('parquet.compress'='SNAPPY');

-- Load partitions from S3 prefixes
MSCK REPAIR TABLE sales_analytics.sales_curated;
```

Step 2 — Run and Compare Queries

```
-- Query 1: Total sales by region (uses partition pruning)
SELECT region, SUM(amount) AS total_sales, COUNT(*) AS order_count
FROM sales_analytics.sales_curated
WHERE region = 'US-EAST' -- partition filter: only scans US-EAST prefix
GROUP BY region
```



```
ORDER BY total_sales DESC;

-- Note the 'Data scanned' shown after query completes

-- Query 2: Top products (column pruning – reads only 2 columns)
SELECT product, SUM(amount) AS revenue
FROM sales_analytics.sales_curated
GROUP BY product
ORDER BY revenue DESC
LIMIT 10;

-- Query 3: CTAS – create pre-aggregated summary table
CREATE TABLE sales_analytics.daily_summary
WITH (format='PARQUET', external_location='s3://your-bucket/curated/daily_summary/')
AS
SELECT order_date, region, SUM(amount) AS daily_revenue
FROM sales_analytics.sales_curated
GROUP BY order_date, region;
```

Step 3 — Create an Athena View

```
-- Create a view for high-value orders
CREATE OR REPLACE VIEW sales_analytics.high_value_orders AS
SELECT order_id, customer_id, product, amount, order_date, region
FROM sales_analytics.sales_curated
WHERE amount > 500;

-- Query the view
SELECT * FROM sales_analytics.high_value_orders
ORDER BY amount DESC;
```

Step 4 — Workgroup Cost Control

1. Go to Athena > Workgroups > Create Workgroup
2. Name: dev-team | Set S3 results location
3. Enable data usage control: limit per query = 1 GB scanned
4. Run a query — it will fail if scan exceeds 1 GB (protecting against expensive scans)

Exam Tip: Athena charges for data scanned, not query time. A query on Parquet/Snappy that prunes partitions can cost 50x less than the same query on unpartitioned CSV. Always convert and partition data before analysis.

TOPIC 4 | Data Formats for AWS Data Engineering

Why Data Format Choice Matters

Choosing the right data format directly impacts query performance, storage cost, serialization speed, and compatibility with downstream services. For the AWS Data Engineer Associate exam, you must know when to use each format and why.

Format Comparison Matrix

Format	Type	Compression	Schema	Best For	Splittable	AWS Use
CSV	Row	gzip/bz2	External	Ingestion, exchange	Yes (bz2)	S3, Glue, Athena
JSON	Row	gzip	Self-desc	APIs, logs, semi-struct	No	S3, Kinesis, DDB
Parquet	Columnar	Snappy/gzip	Self-embed	Analytics, Athena, EMR	Yes	S3, Athena, EMR
ORC	Columnar	Zlib/Snappy	Self-embed	Hive, EMR workloads	Yes	EMR, Glue
Avro	Row	Snappy/gzip	Schema reg.	Streaming, Kafka/Kinesis	Yes	Kinesis, MSK
Delta	Col+Log	Snappy	Schema enf.	ACID, time travel	Yes	EMR, Lake Form.

Row-Oriented vs Columnar Storage

Row-Oriented (CSV, JSON, Avro)	Columnar (Parquet, ORC)
Stores all columns of a row together	Stores all values of a column together
Fast for INSERT / full-row access	Fast for analytics on specific columns
Poor compression (mixed data types per block)	Excellent compression (same type per block)
Full row scanned even for 1-column queries	Only requested columns are read from disk
Best for: transactional systems, streaming, interchange	Best for: analytics, Athena, EMR, Redshift Spectrum

Parquet Deep Dive

- Row Groups: Parquet divides data into row groups (default 128MB). Each row group contains column chunks.
- Column chunks: Each column chunk stores a specific column's data with min/max statistics for predicate pushdown.
- Page: Smallest unit within a column chunk. Data + encoding + optional dictionary encoding.
- Dictionary encoding: Repeat values (e.g., 'US-EAST') stored as integer references — massive compression for low-cardinality columns.
- Predicate pushdown: Athena/Spark read min/max stats from row group metadata and skip groups that cannot match WHERE clause.
- Recommended row group size: 128MB–1GB. Larger = better compression, fewer S3 API calls. Too large = poor parallelism.

Format Selection Decision Guide

Scenario	Recommended Format	Reason
Data ingested from external APIs	JSON or CSV	Human-readable; compatible with any source
Long-term storage for analytics	Parquet + Snappy	Best compression + analytics performance
EMR Hive workloads	ORC	Optimized for Hive; better than Parquet for Hive

Scenario	Recommended Format	Reason
Kafka/Kinesis streaming	Avro	Schema evolution support; schema registry compatible
ACID transactions required	Apache Iceberg (on Parquet)	Row-level updates/deletes; time travel
Redshift external tables	Parquet or ORC	Both supported by Redshift Spectrum
Athena performance-critical	Parquet + Snappy + Partitioned	Minimise scanned data = minimise cost
Cross-platform interchange	Avro	Strong schema definition; language-neutral

LAB 4: Convert CSV to Parquet & Measure Athena Cost Savings

Hands-On Exercise

Objective

Demonstrate the cost and performance difference between querying raw CSV vs Parquet/Snappy in Athena. Measure bytes scanned for each.

Step 1 — Prepare CSV Table in Athena

```
-- Create table over raw CSV (no compression, no partitioning)
CREATE EXTERNAL TABLE IF NOT EXISTS sales_analytics.sales_csv (
  order_id      INT,
  customer_id   STRING,
  product       STRING,
  amount        DOUBLE,
  order_date    STRING,
  region        STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
LOCATION 's3://your-bucket/raw/sales/'
TBLPROPERTIES ('skip.header.line.count'='1');
```

Step 2 — Run Same Query on CSV vs Parquet

```
-- Query A: Run on CSV table
SELECT region, SUM(amount) FROM sales_analytics.sales_csv
WHERE region = 'US-EAST'
GROUP BY region;

-- Note: Data scanned = ENTIRE CSV file

-- Query B: Run on Parquet table (from Lab 1/Lab 3)
SELECT region, SUM(amount) FROM sales_analytics.sales_curated
WHERE region = 'US-EAST' -- partition prune!
GROUP BY region;
```

```
-- Note: Data scanned = ONLY US-EAST partition, Parquet columns only
```

Step 3 — Convert CSV to Parquet using Glue (or CTAS)

Option A: CTAS in Athena (simplest)

```
CREATE TABLE sales_analytics.sales_parquet_v2
WITH (
    format = 'PARQUET',
    parquet_compression = 'SNAPPY',
    partitioned_by = ARRAY['region'],
    external_location = 's3://your-bucket/curated/sales_v2/'
)
AS SELECT order_id, customer_id, product, amount,
        CAST(order_date AS DATE) AS order_date, region
FROM sales_analytics.sales_csv;
```

Option B: Using AWS Glue job (production-grade, supports large datasets)

```
# In your Glue script - add this output section:
from awsglue.dynamicframe import DynamicFrame

# Convert back to DynamicFrame for Glue writer
dyf_out = DynamicFrame.fromDF(df_clean, glueContext, 'dyf_out')

# Write as Parquet with Snappy compression
glueContext.write_dynamic_frame.from_options(
    frame=dyf_out,
    connection_type='s3',
    connection_options={'path': 's3://your-bucket/curated/sales_v2/'},
    format='parquet',
    format_options={'compression': 'snappy'}
)
```

Step 4 — Record Your Results

Fill in the table below after running both queries in Athena:

Query	Format	Data Scanned	Estimated Cost
SUM(amount) WHERE region='US-EAST'	CSV (unpartitioned)	_____ MB	\$ _____
SUM(amount) WHERE region='US-EAST'	Parquet + Snappy (partitioned)	_____ MB	\$ _____
Savings	—	_____ %	_____ %

Expected Result: For typical datasets, Parquet/Snappy with partition pruning reduces scanned data by 85–95% vs unpartitioned CSV. Cost reduction is proportional to bytes scanned.

Day 2 — Summary & Exam Quick Reference

Service	Key Exam Points
AWS Glue	Serverless Spark ETL DynamicFrames Job Bookmarks prevent reprocessing G.1X/G.2X workers Data Catalog = Hive Metastore Triggers: schedule/event/on-demand
EMR	Managed Spark/Hive/Presto Master=On-Demand, Core=cautious Spot, Task=Spot OK EMR Serverless=no cluster management HDFS on Core nodes Use EMR for custom libraries or massive scale
Athena	Serverless SQL on S3 \$5/TB scanned Parquet+Snappy = biggest cost saving Partition pruning Workgroups for cost control CTAS for output Federated Query via Lambda connectors
Data Formats	Parquet/ORC = columnar = analytics CSV/JSON/Avro = row = ingestion/streaming Parquet+Snappy default for analytics Avro for schema evolution in streaming Iceberg for ACID on S3